记忆系统

目录

- 1. 概述与设计哲学
  - 1.1 核心设计原则
  - 1.2 原则二：四点时间戳锚定
  - 1.3 原则三：旁路查询（Side Query）
  - 1.4 原则四：TRUSTING_RECALL_SECTION
  - 1.5 原则五：作用域严格隔离
  - 1.6 extractMemories 协作互斥设计
  - 1.7 记忆用纯文本为介质而非数据库原因（猜测）
- 2. 文件结构全览
  - 2.1 模块目录结构
- 3. 记忆类型体系详解
  - 3.1 四种类型定义
  - 3.2 各类型作用域与保存规范
  - 3.3 Frontmatter 格式规范
  - 3.4 Memory Drift（记忆漂移）警告
  - 3.5 解析函数
- 4. 存储机制：文件系统架构
  - 4.1 目录结构（三层作用域）
  - 4.2 MEMORY.md 索引文件（Entrypoint）
  - 4.3 Auto Memory 目录路径解析
  - 4.4 团队记忆路径
- 5. 核心数据结构
  - 5.1 MemoryHeader（扫描结果）
  - 5.2 RelevantMemory（检索结果）
  - 5.3 MemoryFileInfo（完整记忆文件）
  - 5.4 SyncState（团队记忆同步状态）
  - 5.5 SyncResult 各类响应类型
  - 5.6 SessionMemoryConfig（会话记忆配置）
- 6. 检索与搜索算法
  - 6.1 记忆文件扫描（scanMemoryFiles）
  - 6.2 AI 相关性检索（findRelevantMemories）
  - 6.3 条件规则（Conditional Rules）
- 7. 记忆新鲜度（Staleness）机制
  - 7.1 算法实现
  - 7.2 触发条件
  - 7.3 设计意图
- 8. 会话记忆（Session Memory）系统
  - 8.1 系统定位

- 8.2 提取触发条件
- 8.3 会话记忆模板结构
- 8.4 提取执行流程
- 8.5 会话记忆压缩（Compaction）
- 9. 团队记忆云同步系统
  - 9.1 API 端点
  - 9.2 同步状态机
  - 9.3 完整同步算法
  - 9.4 哈希算法
  - 9.5 文件系统 Watcher
  - 9.6 敏感信息扫描（Secret Scanner）
- 10. Agent 记忆系统
  - 10.1 三种作用域
  - 10.2 快照系统
- 11. @include 指令详解
  - 11.1 语法
  - 11.2 解析流程（claudemd.ts）
  - 11.3 支持的文件类型
  - 11.4 条件 @include（paths 字段）
- 12. 记忆加载顺序与优先级
  - 12.1 四级加载顺序
  - 12.2 内容合并策略
- 13. 安全机制
  - 13.1 路径遍历防护
  - 13.2 配置注入防护
  - 13.3 敏感信息防护
  - 13.4 文件权限
- 14. 关键函数完整索引
  - 14.1 自动记忆模块（memdir/）
  - 14.2 会话记忆模块（services/SessionMemory/）
  - 14.3 团队记忆同步模块（services/teamMemorySync/）
  - 14.4 Agent 记忆模块（tools/AgentTool/）
  - 14.5 核心工具模块（utils/）
  - 14.6 Auto-Dream 模块（services/autoDream/）
  - 14.7 记忆提取模块（services/extractMemories/）
- 15. 系统架构总图
  - 15.1 架构主图
  - 15.2 后台进程图
- 16. 算法复杂度分析
  - 16.1 操作复杂度表
- 17. Auto-Dream 系统（自动记忆整合）
  - 17.1 系统定位

    - 17.2 锁文件机制（consolidationLock.ts）
    - 17.3 四阶段整合 Prompt（consolidationPrompt.ts）
    - 17.4 工具约束
    - 17.5 DreamTask UI 集成
    - 17.6 GrowthBook 配置与开关
    - 17.7 与 /dream 技能的关系
- 18. ExtractMemories 系统（会话末记忆提取）
    - 18.1 系统定位
    - 18.2 Forked Agent 模式
    - 18.3 闭包状态与 Trailing Run
    - 18.4 提交流水线
    - 18.5 提取 Prompt 模板
- 19. DreamTask（梦境任务 UI 集成）
    - 19.1 设计背景
    - 19.2 任务状态结构
    - 19.3 消息监听（makeDreamProgressWatcher）
    - 19.4 杀任务（kill）
- 20. 每日日志模式（Assistant Mode / KAIROS）
    - 20.1 背景
    - 20.2 与标准模式的区别
- 21. 启动链路与模块初始化
    - 21.1 backgroundHousekeeping.ts
    - 21.2 stopHooks.ts 触发链路
    - 21.3 关闭时 drain
- 附录 A：Zod Schema 定义
    - A.1 Schema 定义
- 附录 B：全局状态一览
    - B.1 各模块全局状态

---

1. 概述与设计哲学

Claude Code 的记忆系统是一套**分层、文件化、长期持久化**的架构，采用纯文本（Markdown + YAML frontmatter）作为存储介质，而非数据库

PS：
YAML frontmatter = 记忆的"索引卡"

- name、description、type — 用来找到这条记忆、判断是否相关
  Markdown body = 记忆的"正文内容"
- 实际要记住的信息本身

关系：frontmatter 是附着在 Markdown 文件顶部的元数据，两者写在同一个 .md 文件里

	YAML frontmatter	Markdown body
目的	检索、分类、相关性判断	存储实际内容
格式	结构化键值对	自然语言文本
使用时机	加载时快速扫描、 relevance 过滤	命中介入时才作为上下文读入

1.1 核心设计原则

原则一：可推导信息不保存 -> 即可以直接捡回来的信息

以下内容明确定义为"不应存入记忆"：

- 代码模式、约定、架构（可从项目当前状态推导）
- Git 历史（使用 `git log` / `git blame`）
- 调试解决方案（修复代码本身才是权威）
- CLAUDE.md 中已有的内容
- 临时任务细节

> **源码中的 eval-validated 补充：** `memoryTypes.ts` 中 `WHAT_NOT_TO_SAVE_SECTION` 最后一段：
>
> ```
> // H2: explicit-save gate. Eval-validated (memory-prompt-iteration case 3,
> // 0/2 → 3/3): prevents "save this week's PR list" → activity-log noise.
> 'These exclusions apply even when the user explicitly asks you to save.
>  If they ask you to save a PR list or activity summary,
>  ask what was *surprising* or *non-obvious* about it — that is the part worth keeping.'
> ```
>
> 这是一条**eval 驱动的设计决策**：在没有这条规则时，模型会把"用户要求的保存"直接执行，导致 activity-log 噪音。核心洞见：**用户请求保存 ≠ 应该保存**，只有"保存什么"背后的洞见才值得保留。

1.2 原则二：四点时间戳锚定

每条记忆在文件中附带 `mtimeMs` （修改时间），系统据此计算新鲜度。超过 1 天的记忆会触发"过时警告"，要求使用者验证当前代码状态。

```
// memoryAge.ts
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''                          // 今天/昨天 → 无提示
  if (d <= 7) return `This memory is ${d} days old.`
  if (d <= 30) return `This memory is ${d} days old. Verify before citing.`
  // 30天以上 → stale reasoning 文本
}
```

> **设计原理：** `memoryAge.ts` 注释说明：
> *"Models are poor at date arithmetic — a raw ISO timestamp doesn't trigger staleness reasoning the way '47 days ago' does."*

> 语言模型对 ISO 时间戳的算术推理能力较差，直接给出"47 天前"比给出"2026-02-12T..."更能触发模型对记忆过时的警觉。此外，警告文案刻意强调"point-in-time observation"——**一条记忆是对某个时间点状态的快照，不代表当前状态**，这对包含文件:行号引用的记忆尤其重要（引用可能早已失效）。

1.3 原则三：旁路查询（Side Query）

相关性检索使用 **Sonnet 模型进行独立查询**，不消耗主对话上下文 token，也不影响主对话的推理流。

```
// SELECT_MEMORIES_SYSTEM_PROMPT 关键设计片段
const SELECT_MEMORIES_SYSTEM_PROMPT = `...
- If a list of recently-used tools is provided, do not select memories that
  are usage reference or API documentation for those tools
  (Claude Code is already exercising them).
  DO still select memories containing warnings, gotchas, or known issues
  about those tools — active use is exactly when those matter.`
```

这里有一个精确的语义区分：**工具使用文档不选**（对话里已经有了），但**工具的"警告/gotcha/已知问题"要选**——正在使用中，警告信息此刻最有用。通过 `recentTools` 参数传入最近使用的工具列表，selector 会排除这些工具的 API 参考文档，同时保留活跃使用场景下的警告类记忆。

> **Telemetry 钩子**（`findRelevantMemories.ts`，受 `feature('MEMORY_SHAPE_TELEMETRY')` 控制）：
> 模型选择结果与扫描结果被送入 `logMemoryRecallShape`，记录 `selection_rate` 等指标。**选择集为空时仍然发送数据**，分母（`memories.length`）始终存在，所以能区分"检索运行了但没选到"和"检索根本没运行"，这是两个完全不同的产品信号。

> **PS：记忆检索是异步的？放到上下文中的时序是否会有问题？**
>
> 解答：这个 prefetch 模式在"不阻塞"和"尽量赶本轮"之间做了务实的折中——用 polling 代替阻塞等待，允许 prefetch 晚于 API 调用开始，但尽力让它在本轮内完成（**API请求发出之前极有可能赶不上，但是尽量赶上主流程的LLM本轮流式返回之前**）。如果要严格保证"检索必须同步"，需要把 prefetch 改成 await 在 for await 之前，但这会引入一个额外的 RTT 延迟

1.4 原则四：TRUSTING_RECALL_SECTION——记忆召回后的信任层次

这是 `memoryTypes.ts` 中一段**eval-validated 的设计**，核心理念是：**找到一条记忆 ≠ 可以信任这条记忆的内容**。

```
// memoryTypes.ts
export const TRUSTING_RECALL_SECTION: readonly string[] = [
  '## Before recommending from memory',
  'A memory that names a specific function, file, or flag is a claim that ' +
    'it existed *when the memory was written*. It may have been renamed, ' +
    'removed, or never merged. Before recommending it:',
  '- If the memory names a file path: check the file exists.',
  '- If the memory names a function or flag: grep for it.',
  '- If the user is about to act on your recommendation, verify first.',
  '"The memory says X exists" is not the same as "X exists now."',
]
```

1.5 原则五：作用域严格隔离

四种记忆类型各自有隐含的默认作用域（私有或团队），且配置注入路径有安全限制，防止恶意项目配置将记忆写入敏感目录。

1.6 extractMemories 协作互斥设计——主 Agent 与 Forked Agent 的分工

`extractMemories.ts` 的最核心设计理念：**不是"两套系统竞争"，而是"主 Agent 先写，Forked Agent 补漏"**。

```
// extractMemories.ts
/**
 * Returns true if any assistant message after the cursor UUID contains a
 * Write/Edit tool_use block targeting an auto-memory path.
 *
 * The main agent's prompt has full save instructions — when it writes
 * memories, the forked extraction is redundant. runExtraction skips the
 * agent and advances the cursor past this range, making the main agent
 * and the background agent mutually exclusive per turn.
 */
function hasMemoryWritesSince(
  messages: Message[],
  sinceUuid: string | undefined,
): boolean {
  // ...检查主 agent 是否写过 auto-memory 路径
}
```

协作流程：

1. 主 Agent 在对话中**主动写了记忆**（受完整系统提示词指导）→ forked agent 检测到后，**跳过提取但推进游标** → 主 Agent 胜出。
2. 主 Agent 在对话中**没有写记忆** → forked agent 启动，用独立的提取 prompt 分析最近的对话，补充记忆。

> **为什么这样设计？** 主 Agent 的系统提示词有完整的记忆保存说明，它写的记忆是"有意为之"的（基于当前对话语义）。Forked agent 是"无差别扫描"（基于最近 N 条消息的结构化提取）。两者各有优势：**主 Agent 写的是洞见**（为什么这样做），**Forked Agent 补的是事实**（发生了什么）。合并起来才能形成完整的记忆。

Throttle + Trailing Run 的双重保险：

```
// extractMemories.ts
// 每个 N 个 eligible turn 才运行一次提取（tengu_bramble_lintel，默认 1）
turnsSinceLastExtraction++
if (turnsSinceLastExtraction < getFeatureValue_CACHED('tengu_bramble_lintel', 1)) {
  return  // 跳过本次
}

// 如果调用到达时正在运行，把上下文 stash 起来
if (inProgress) {
  pendingContext = { context, appendSystemMessage }  // trailing run
  return
}

// trailing run：处理两次调用之间积累的新消息
const trailing = pendingContext
```

```
if (trailing) {
  await runExtraction({ context: trailing.context, isTrailingRun: true })
}
```

- **Trailing Run 设计**：主提取运行期间，用户可能又发了好几条消息。Stash 最新上下文，运行完当前提取后立即用新消息跑一次 trailing，确保不漏。
- **Throttle 设计**（`tengu_bramble_lintel`）：避免每轮都触发提取，默认值为 1（每轮都可能触发，但受其他条件限制）。

5 turn 硬上限：提取 agent 最多运行 5 轮就会被强制终止。

> 源码注释：`"Well-behaved extractions complete in 2-4 turns (read → write). A hard cap prevents verification rabbit-holes from burning turns."`
>
> 防止 Agent 陷入"读取→验证→再读取→再验证"的兔子洞，消耗过多 token。实践中 2-4 轮足够完成一次有效提取。

内存扫描的"单遍读-后排序"优化（`scanMemoryFiles.ts`）：

```
// memoryScan.ts
/**
 * Single-pass: readFileInRange stats internally and returns mtimeMs, so we
 * read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
 * this halves syscalls vs a separate stat round; for large N we read a few
 * extra small files but still avoid the double-stat on the surviving 200.
 */
```

- 常见场景（N ≤ 200）：只读前 30 行 frontmatter，一次遍历搞定
- 大 N 场景：最多读 200 个文件的 frontmatter，仍然只读 200 个（而不是 stat 所有文件后再选 200 个）。
- `readFileInRange` 内部同时返回 content 和 mtime，所以 stat 整合在读取路径里。

ensureMemoryDirExists 的"预承诺"设计（`memdir.ts`）：

```
// memdir.ts
/**
 * Shared guidance text appended to each memory directory prompt line.
 * Shipped because Claude was burning turns on `ls`/`mkdir -p` before writing.
 * Harness guarantees the directory exists via ensureMemoryDirExists().
 */
export const DIR_EXISTS_GUIDANCE =
  'This directory already exists — write to it directly with the Write tool ' +
  '(do not run mkdir or check for its existence).'
```

设计意图：如果不预建目录，模型在每次写记忆前会消耗 1-2 个 turn 去检查/创建目录（burn turns）。通过 `ensureMemoryDirExists`（在 `loadMemoryPrompt` 入口保证调用一次）提前建立目录，再把"目录已存在，不需要 mkdir"作为 prompt 指导直接告知模型，节省下来的 turn 可以用于实际工作。

1.7 记忆用纯文本为介质而非数据库原因（猜测）

1. 天然兼容 Git 工作流，实现“代码与认知”的版本对齐

纯文本文件天然支持 Git 差异对比、版本历史追踪和 PR 审查。如果使用数据库，Git 只能存储二进制差异，无法进

行有意义的代码审查和变更对比

2. 人类可读性与可干预性
采用纯文本格式，开发者可以直接在编辑器中打开，像修改普通代码一样直接阅读、纠错或补充上下文

3. 完全对齐 Agent 的认知模型与原生能力
纯文本（如 Markdown）是大语言模型最擅长处理的格式。给它一个路径，它就能直接读写和编辑。如果引入数据库，Agent 还需要掌握数据库连接、建表、结构迁移、SQL 查询等额外能力，增加了不必要的复杂度

4. 无额外基础设施与部署成本，降低跨环境同步门槛
纯文本文件不依赖任何外部服务，是最简单、最轻量的持久化方案。同时，在团队协作中，**纯文本记忆文件可以直接随代码仓库拉取到新成员的本地环境，实现真正的“开箱即用”**。

5. 得益于底层模型超长上下文窗口的普及

2. 文件结构全览

2.1 模块目录结构

```
src/
├── memdir/                                  # ★ 自动记忆核心模块
│   ├── memoryTypes.ts                       # 记忆类型枚举、frontmatter 解析
│   ├── memoryScan.ts                        # 记忆文件扫描、manifest 格式化
│   ├── memoryAge.ts                         # 记忆新鲜度计算
│   ├── memdir.ts                            # 记忆 prompt 构建、MEMORY.md 截断
│   ├── paths.ts                             # 记忆路径解析、自动记忆开关
│   ├── teamMemPaths.ts                      # 团队记忆路径安全验证
│   ├── teamMemPrompts.ts                    # 团队记忆 prompt 组合
│   └── findRelevantMemories.ts              # AI 相关性检索
│
├── services/
│   ├── SessionMemory/                       # ★ 会话记忆模块
│   │   ├── sessionMemory.ts                # 提取触发逻辑、REPL hook
│   │   ├── sessionMemoryUtils.ts            # 全局状态、配置、等待函数
│   │   └── prompts.ts                      # 会话记忆模板、更新 prompt
│   ├── teamMemorySync/                      # ★ 团队记忆云同步模块
│   │   ├── index.ts                        # pull/push/sync 主逻辑
│   │   ├── watcher.ts                      # fs.watch 防抖推送
│   │   ├── secretScanner.ts                # 上传前敏感信息扫描
│   │   └── types.ts                        # Zod schema、API 响应类型
│   ├── compact/
│   │   └── sessionMemoryCompact.ts         # 基于会话记忆的上下文压缩
│   ├── autoDream/                           # ★ Auto-Dream 后台整合模块
│   │   ├── autoDream.ts                    # 三门触发 + forked agent 执行
│   │   ├── config.ts                       # isAutoDreamEnabled() 开关
│   │   ├── consolidationLock.ts            # 锁文件（mtime = lastConsolidatedAt）
│   │   └── consolidationPrompt.ts          # 四阶段整合 prompt
│   └── extractMemories/                     # ★ 会话末记忆提取模块
│       ├── extractMemories.ts               # forked agent 提取主逻辑
│       └── prompts.ts                       # 提取 prompt 模板
│
├── tasks/DreamTask/
│   └── DreamTask.ts                         # DreamTask UI 集成（footer pill）
│
```

```
├── tools/AgentTool/
│   ├── agentMemory.ts                      # Agent 级别持久记忆
│   └── agentMemorySnapshot.ts              # 快照初始化与同步
│
├── components/memory/
│   ├── MemoryFileSelector.tsx              # 记忆文件选择 UI 组件
│   └── MemoryUpdateNotification.tsx       # 记忆更新通知组件
│
├── commands/memory/
│   └── memory.tsx                          # /memory CLI 命令实现
│
└── utils/
    ├── claudemd.ts                         # CLAUDE.md 加载主逻辑、@include 解析
    └── memoryFileDetection.ts              # 记忆文件类型自动检测
```

---

3. 记忆类型体系详解

3.1 四种类型定义

定义于 `src/memdir/memoryTypes.ts`，采用 **const assert 闭集枚举**：

```
export const MEMORY_TYPES = [
  'user',        // 用户偏好、角色、专业知识
  'feedback',    // 工作指导（避免/重复的行为模式）
  'project',     // 进行中工作、目标、倡议 initiative
  'reference'    // 外部系统的引用/指针
] as const

export type MemoryType = (typeof MEMORY_TYPES)[number]
// 推导结果: 'user' | 'feedback' | 'project' | 'reference'
```

3.2 各类型作用域与保存规范

类型	默认作用域	何时升为团队级	保存内容示例
`user`	私有（Always privat…	永不	用户是高级 Go 工程师，首次接触 React
`feedback`	私有（默认）	仅当团队共识需要	集成测试必须连接真实数据库（mock 导致生产事故）
`project`	团队（强烈偏向）	通常就是团队	"INGEST" Linear 项目跟踪所有管道 bug
`reference`	团队（通常）	通常就是团队	Grafana 面板 grafana.internal/d/api-latency 是 oncall 监控

PS：私人记忆（user / feedback / project / reference）只存在本地磁盘；只有 **Team Memory**（团队共享部分）会同步到云端，供同一 repo 的组织成员共享

3.3 Frontmatter 格式规范

每条记忆为一个 Markdown 文件，必须包含 YAML frontmatter：

PS：采用标准的 YAML frontmatter（--- 分隔符），直接正则匹配出来的

```
---
name: {{记忆唯一名称 — 使用 snake_case 或 kebab-case}}
```

```
description: {{一句话描述 — 用于 AI 判断相关性，越具体越好}}
type: {{user | feedback | project | reference}}
---

记忆正文内容。

对于 feedback/project 类型，建议格式：
规则/事实本身
**Why:** 原因说明 — 通常是过去的事件或strong偏好
**How to apply:** 何时何地应用此规则
```

> **YAML 特殊字符自动引用设计**（`frontmatterParser.ts`）：
>
> 源码注释解释了为什么需要这个设计：
>
> ```
> const YAML_SPECIAL_CHARS = /[{}[]*&#!|>%@`]|: /
> // - { } are flow mapping indicators
> // - * is anchor/alias indicator
> // - [ ] are flow sequence indicators
> // - ': ' (colon followed by space) is key indicator
> //   causes 'Nested mappings are not allowed in compact mappings'
> // - # is comment indicator
> ```
>
> 典型问题场景：`paths: "**\\/*.{ts,tsx}"` 中的 `*`、`{`、`}`、`/` 都是 YAML 特殊字符，如果不做处理直接写进 frontmatter，YAML 解析器会报错。
>
> `quoteProblematicValues()` 在解析失败时会自动重试，将所有含特殊字符的 value 自动用双引号包裹。写入时也已预引用，保证来回一致性。

3.4 Memory Drift（记忆漂移）警告

当记忆内容与当前代码状态冲突时，系统要求：（按照时效性，当前情况优先级大于记忆）

> 信任观察到的现状（what you observe now），而非记忆中的记录——更新或删除过时记忆，不要基于过时信息做出断言。

3.5 解析函数

```
// memoryTypes.ts
export function parseMemoryType(raw: unknown): MemoryType | undefined {
  // 无效或缺失值返回 undefined，不抛出异常
  if (typeof raw !== 'string') return undefined
  // 第二行：在 ['user', 'feedback', 'project', 'reference'] 里找精确匹配
  return MEMORY_TYPES.find(t => t === raw)
}
```

---

4. 存储机制：文件系统架构

4.1 目录结构（三层作用域）

```
~/.claude/                                  # 用户级（全局私有）
├── memory/
```

```
│   ├── MEMORY.md                          # 索引清单（入口点）
│   ├── user_role.md                       # 记忆文件 *.md
│   ├── feedback_testing.md
│   ├── project_ingest.md
│   ├── reference_linear.md
│   └── team/                              # 团队共享记忆
│       ├── MEMORY.md
│       ├── *.md
│
├── agent-memory/                          # Agent 持久记忆（用户作用域）
│   └── <agentType>/
│       └── MEMORY.md
│
└── agent-memory-local/                    # Agent 持久记忆（本地作用域）
    └── <agentType>/
        └── MEMORY.md

<git-root>/.claude/                        # 项目级（项目内共享）
├── memory/
│   ├── MEMORY.md
│   ├── *.md
│   └── team/
│       ├── MEMORY.md
│       └── *.md
│
└── agent-memory/                          # Agent 持久记忆（项目作用域）
    └── <agentType>/
        └── MEMORY.md
```

区分：

```
~/.claude/projects/<project>/memory/ — 用户 Auto Memory

- 作用域：用户 × 项目（跨会话）
- 路径：~/.claude/projects/<sanitized-git-root>/memory/
- 性质：用户个人的记忆，存在于用户配置目录下
- 可见性：只有该用户自己能看到
- 团队共享：不共享（不在项目目录内）
- 记忆范围：user / feedback / project / reference 四类，这是 prefetch 和 findRelevantMemories 操作的主要目录

~/.claude/agent-memory/<agentType>/ — Agent Memory (user scope)

- 作用域：agent × 用户全局（跨所有项目）
- 路径：~/.claude/agent-memory/<agentType>/
- 性质：agent 的持久化记忆，跟 agent 类型走，不跟项目走
- 可见性：只有该 agent 自己能看到
- 团队共享：不共享

<cwd>/.claude/agent-memory/<agentType>/ — Agent Memory (project scope)

- 作用域：agent × 当前项目（跨会话）
- 路径：在项目根目录的 .claude/agent-memory/<agentType>/
- 性质：agent 记忆，跟项目走，会被提交到 git
- 可见性：团队所有成员共享同一 agent 实例的上下文时可见

<cwd>/.claude/agent-memory-local/<agentType>/ — Agent Memory (local scope)

- 作用域：agent × 当前机器 × 当前项目
- 路径：在项目目录，gitignored（.claude 通常在 gitignore 里）
- 性质：本地专用，不同步到 VCS
- 可见性：只有当前机器的当前项目能访问
```

4.2 MEMORY.md 索引文件（Entrypoint）

性质：MEMORY.md 是索引清单（manifest），不是记忆容器。这与其他系统（如 Obsidian）不同——不应当把内容直接写在 MEMORY.md 中。

格式规范：

```
- [Title](filename.md) — 一句话描述（可选）
- [user] user_role.md (2026-03-30T10:00:00.000Z): 用户是数据科学家
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
```

硬性限制（`memdir.ts`）：

- 最大200 行
- 最多25,000 字节

超出限制时，`truncateEntrypointContent()` 同时应用行数和字节数双重截断，并记录 `wasLineTruncated` 和 `wasByteTruncated` 标志。

截断后会追加警告

```
> WARNING: MEMORY.md is 347 lines and 94KB (limit: 200 lines and 25KB).
    Only part of it was loaded. Keep index entries to one line under ~200 chars;
    move detail into topic files.
```

PS：信息不是完全丢掉了，只是MEMORY.md中索引丢掉了。因此即使一个 entry 被截掉了，如果用户问到相关内容，Sonnet 相关性选择器仍然能从磁盘读到那个 `topic` 文件的 `frontmatter` 并把它捞出来

4.3 Auto Memory 目录路径解析

完整路径解析（getAutoMemPath()，优先级从高到低）：

```
  1. CLAUDE_COWORK_MEMORY_PATH_OVERRIDE（环境变量）→ 直接返回完整路径，不拼接
  2. autoMemoryDirectory 配置项（settings.json，设置的优先级：policySettings > flagSettings >
localSettings > userSettings）
    - ⚠ 项目级 .claude/settings.json 禁止设置此字段（安全防护：防止恶意仓库将记忆路径重定向到
  ~/.ssh 等敏感目录）
  3. 默认路径：
  <memoryBase>/projects/<sanitized-git-root>/memory/
    - sanitizePath()：将 git root 路径转换为安全目录名（非字母数字字符替换为 -，超长截断并附加
  hash）
    - getAutoMemBase()：使用 git root 而非 CWD，确保同一 repo 的所有 worktree 共享同一记忆目录

  协作共享：
  - 同一 git root 下的所有工作区共享同一 auto memory
  - team memory 在此基础上额外增加 <autoMemPath>/team/ 目录（通过 TEAMMEM feature 分支）
```

4.4 团队记忆路径

`src/memdir/teamMemPaths.ts` — `getTeamMemPath()`：

```
<memoryBase>/projects/<sanitized-project-root>/memory/team/
```

PS：
这个team memory是统计在服务器侧的状态信息之类的，即有权限的人都可以获取的公共信息

```
Team Memory 共享机制整理

1. 核心定义与粒度
Team Memory 旨在实现同一 Git 仓库所有协作者之间的记忆共享。
```

```
识别粒度：以 git remote URL (owner/repo) 为唯一识别标准。
非显式定义：不依赖传统的“团队”设置，而是基于仓库访问权限。

2. 访问与认证
准入条件：任何具备该仓库访问权限的成员。
身份校验：通过 OAuth 认证 获取令牌，实现对共享记忆的读写。

3. 同步机制 (Pull-Push 模式)
系统通过以下流程确保数据在不同设备间的一致性：
会话开始：执行 Pull 操作，从服务器拉取最新版本并覆盖本地数据。
会话结束：执行 Push 操作，将本地生成的增量改动同步回服务器。

4. 冲突处理与安全保障
冲突管理：引入 ETag 乐观锁 机制，有效解决多人并发编辑时的数据冲突。
隐私防护：上传前执行文件扫描。若检测到 API Key、Token 等敏感凭证，系统将自动跳过该文件，防止敏感信息外泄。

5. 技术特性
独立性：不占用 Git 版本历史，不依赖本地文件系统共享。
云端驱动：纯粹依靠 API 和 OAuth Token 实现跨设备同步。
```

---

5. 核心数据结构

5.1 MemoryHeader（扫描结果）

```ts
// src/memdir/memoryScan.ts
export type MemoryHeader = {
  filename: string            // 文件名（不含路径）
  filePath: string           // 绝对路径
  mtimeMs: number            // 文件修改时间（毫秒 Unix epoch）
  description: string | null // frontmatter 中的 description（用于 manifest）
  type: MemoryType | undefined // frontmatter 中的 type（可缺失）
}
```

用途： `scanMemoryFiles()` 的返回类型，用于 manifest 格式化和**相关性检索**。**只读取文件前 30 行 frontmatter，不读取完整内容**。

5.2 RelevantMemory（检索结果）

```ts
// src/memdir/findRelevantMemories.ts
export type RelevantMemory = {
  path: string     // 记忆文件的绝对路径
  mtimeMs: number  // 修改时间（用于新鲜度计算）
}
```

检索过程详见 6. 检索与搜索算法

5.3 MemoryFileInfo（完整记忆文件）

```ts
// src/utils/claudemd.ts
export type MemoryFileInfo = {
  path: string                          // 文件绝对路径
```

```
  type: MemoryType                    // 'Managed' | 'User' | 'Project' | 'Local'
                                       // 'AutoMem' | 'TeamMem'
  content: string                     // 解析后的纯文本内容
  parent?: string                      // 包含此文件的父文件路径（用于 @include 追踪）
  globs?: string[]                    // frontmatter 中的 paths 字段（条件规则）
  contentDiffersFromDisk?: boolean    // 内存内容是否与磁盘不同步
  rawContent?: string                 // 原始 Markdown 内容（含 frontmatter）
}
```

5.4 SyncState（团队记忆同步状态）

```
// src/services/teamMemorySync/index.ts
export type SyncState = {
  lastKnownChecksum: string | null     // ETag，用于 HTTP 条件请求（If-None-Match）；用于下次 pull
的 If-None-Match（整库级别判断有没有变）

  serverChecksums: Map<string, string> // key（相对路径）-> SHA-256 哈希；用于本次 push 的 delta
计算（文件级别判断哪些变了）

  serverMaxEntries: number | null      // 服务器返回的最大条目数限制（从 413 响应学习）；用于 push 前的
本地截断（避免触发 413）
}
```

PS：详见 [9. 团队记忆云同步系统]

- 为了同步服务器上最新的团队记忆
- 团队记忆存在服务器上，本质是一个共享的记忆数据库。每个成员的本地只是一个缓存副本——pull 是同步最新，push 是把自己的改动合并上去

```
  服务器对整个 team memory 快照计算了一个 SHA-256 哈希 摘要，每次 push 成功后服务器返回新的 checksum。
  用途是条件请求：

  本地 pull 时：
    请求头带上 If-None-Match: "<上次的 checksum>"
    服务器比较：
      checksum 没变 → 返回 304 Not Modified（没变化，不下载任何内容）
      checksum 变了 → 返回完整数据 + 新 checksum

  这样每次 pull 不需要下载完整内容，只有真正有变化时才传数据
```

5.5 SyncResult 各类响应类型

```
// src/services/teamMemorySync/types.ts
export type TeamMemorySyncFetchResult = {
  success: boolean
  data?: TeamMemoryData
  isEmpty?: boolean
  notModified?: boolean        // HTTP 304，等效于上次
  checksum?: string            // ETag
  error?: string
  skipRetry?: boolean          // 永久失败，跳过重试
  errorType?: 'auth' | 'timeout' | 'network' | 'parse' | 'unknown'
```

```
  httpStatus?: number
}

export type TeamMemorySyncPushResult = {
  success: boolean
  filesUploaded: number
  checksum?: string
  conflict?: boolean          // HTTP 412，需要刷新哈希重试
  error?: string
  skippedSecrets?: SkippedSecretFile[] // 跳过含敏感信息的文件
  errorType?: 'auth' | 'timeout' | 'network' | 'conflict' |
              'unknown' | 'no_oauth' | 'no_repo'
  httpStatus?: number
}
```

PS：

- 团队共享知识库的文件同步服务，个服务就是负责把本地文件上传到服务器（push）和从服务器拉取（pull）
- **上传记忆之前会进行脱敏**，详见9.6 敏感信息扫描（Secret Scanner）

5.6 SessionMemoryConfig（会话记忆配置）

```
// src/services/SessionMemory/sessionMemoryUtils.ts
export type SessionMemoryConfig = {
  minimumMessageTokensToInit: number      // 开始初始化的最小 token 数
  minimumTokensBetweenUpdate: number     // 两次更新之间的最小增量 token 数
  toolCallsBetweenUpdates: number        // 两次更新之间的最小工具调用次数
}

export const DEFAULT_SESSION_MEMORY_CONFIG: SessionMemoryConfig = {
  minimumMessageTokensToInit: 10000,
  minimumTokensBetweenUpdate: 5000,
  toolCallsBetweenUpdates: 3,
}
```

---

6. 检索与搜索算法

6.1 记忆文件扫描（scanMemoryFiles）

文件： `src/memdir/memoryScan.ts`

```
export async function scanMemoryFiles(
  memoryDir: string,
  signal: AbortSignal,
): Promise<MemoryHeader[]>
```

算法步骤：

1. 调用 `readdir(memoryDir, { recursive: true })` 遍历目录
2. 过滤出所有 `.md` 文件（排除 MEMORY.md 自身）
3. 对每个文件：
   - 调用 `readFileInRange(filePath, { maxLines: 30 })` 只读前 30 行 -> 对应 frontmatter

    - 调用 `parseFrontmatter(raw)` 解析 frontmatter
    - `readFileInRange` 内部同时返回 `mtimeMs`（无需单独 stat）
4. 提取 `{ filename, filePath, mtimeMs, description, type }`
5. 按 `mtimeMs` 降序排序（最新在前）
6. 截断至最多 200 个文件（`MAX_MEMORY_FILES`）
7. 返回 `MemoryHeader[]`

> **单遍读-后排序优化**：源码注释（`memoryScan.ts`）解释了这一设计：
>
> ```
> // Single-pass: readFileInRange stats internally and returns mtimeMs, so we
> // read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
> // this halves syscalls vs a separate stat round; for large N we read a few
> // extra small files but still avoid the double-stat on the surviving 200.
> ```

Manifest 格式化：

```
export function formatMemoryManifest(memories: MemoryHeader[]): string {
  return memories.map(m => {
    const tag = m.type ? `[${m.type}] ` : ''
    const ts = new Date(m.mtimeMs).toISOString()
    return m.description
      ? `- ${tag}${m.filename} (${ts}): ${m.description}`
      : `- ${tag}${m.filename} (${ts})`
  }).join('\n')
}
```

输出示例：

```
- [user] user_role.md (2026-03-30T10:00:00.000Z): 用户是数据科学家，专注可观测性
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
- [project] project_ingest.md (2026-03-28T15:30:00.000Z): Linear 项目 INGEST 跟踪管道 bug
```

6.2 AI 相关性检索（findRelevantMemories）

文件：`src/memdir/findRelevantMemories.ts`

这是记忆系统的核心智能层，使用 Sonnet 模型进行旁路查询。

```
export async function findRelevantMemories(
  query: string,                    // 当前用户消息/问题
  memoryDir: string,                 // 记忆目录
  signal: AbortSignal,
  recentTools: readonly string[] = [],  // 最近使用的工具（上下文信号）
  alreadySurfaced: ReadonlySet<string> = new Set(), // 已展示过的记忆（去重）
): Promise<RelevantMemory[]>
```

算法步骤：

```
Step 1: 调用 scanMemoryFiles() 获取所有记忆的 manifest
Step 2: 格式化 manifest（带时间戳和描述）
Step 3: 构建 System Prompt（SELECT_MEMORIES_SYSTEM_PROMPT）:
```

```
        "You are a memory retrieval assistant. Given a user query and a list of
         available memories, select up to 5 memories most relevant to the query.
         Prefer memories with recent timestamps and direct topic matches.
         Output JSON: { "selected_memories": ["filename1", "filename2", ...] }
         Exclude: memories already surfaced in this conversation."

Step 4: 使用 Sonnet 模型（sideQuery）发送请求
Step 5: 解析 JSON 响应，提取 selected_memories 数组
Step 6: 过滤：排除 alreadySurfaced 中已展示过的记忆
Step 7: 返回 RelevantMemory[]（path + mtimeMs）
```

关键设计：Side Query 意味着相关性检索在独立调用中进行，不消耗主对话 token，不影响主推理流。

6.3 条件规则（Conditional Rules）

`claudemd.ts` 支持 frontmatter 中的 `paths` 字段，用于条件化记忆：

```
---
name: vscode-settings-hint
description: VSCode 特殊设置提示
type: user
paths:
  - "**/.vscode/**"
  - "**/settings.json"
---

当用户编辑 VSCode 配置时……
```

系统通过 `glob` 模式匹配当前工作目录路径，仅在匹配时注入该记忆。

PS：
典型的 **上下文感知** 或者说 **动态提示词注入** 设计，简单来说：

- 普通记忆是"全局生效"的——只要加载了，就一直存在于上下文里
- 条件规则允许记忆文件声明：**只有当我编辑特定路径的文件时，这条记忆才有用，从而只在匹配时注入**
  个人猜测这是个很强的功能，因为和路径绑定，可以实现以下效果：
- 不同项目、不同文件类型、不同工作阶段，自动切换不同的行为规范和提示词，而不需要用户手动指定
- 团队可以在 team memory 里声明项目宪法，每个成员进入工作区时 Claude Code 自动以这套宪法为基准行事
- 采用惰性加载的思路，这个功能变成了一个激活和加载记忆的时机，如果路径下是skill那么这里就可以放该skill对应的记忆，如果是MCP工具，就可以存该MCP工具记忆，如果是某个工程文件就会变成这个工程的本地记忆。这种策略可以存多种多样的记忆而且不会导致大量记忆索引条目引起的上下文问题

---

7. 记忆新鲜度（Staleness）机制

文件：`src/memdir/memoryAge.ts`

7.1 算法实现

```
// 以毫秒为单位的每天：86_400_000
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}
```

```
// 计算公式：(当前时间 - 文件修改时间) / 每天毫秒数，向下取整得天数
// 兜底 Math.max(0, ...) 处理时钟偏移导致的负数（文件比当前时间还"新"），clamp 至 0
// 返回值示例：今日: 0 | 昨日: 1 | 47天前: 47

// 生成记忆新鲜度警告文本，用于提醒模型该记忆可能已过时
export function memoryAge(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d === 0) return 'today'
  if (d === 1) return 'yesterday'
  return `${d} days ago`
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''  // 今日/昨日无警告
  return (
    `This memory is ${d} days old. ` +
    `Memories are point-in-time observations, not live state — ` +
    `claims about code behavior or file:line citations may be outdated. ` +
    `Verify against current code before asserting as fact.`
  )
}
```

7.2 触发条件

年龄	行为
0 天（今日）	无特殊处理
1 天（昨日）	无特殊处理
≥2 天	在 `<system-reminder>` 标签中附加过时警告

注：`system-reminder`

`<system-reminder>` 是 Claude Code 中一个精心设计的 per-turn 上下文注入机制，核心目标是：在每次对话轮次中向模型**注入动态信息**，而不污染静态 system prompt 本身

简单类比:
可以把 `<system-reminder>` 理解为模型视角的“便签”系统：

- 逻辑依附：贴在任意工具结果旁边（逻辑上）。
- 补充提醒：告知模型“与当前工具无关但你应该知道的事”(如记忆陈旧度、文件风险等)。
- 无感交互：对用户 UI 不可见（前端会过滤/隐藏掉）。
- 轻量高效：比修改 system prompt 更轻量，无需重新构建或缓存巨大的静态前缀。

注入点：

- **记忆陈旧度提醒 (`memoryAge.ts`)**：为旧记忆（超过1天）自动附上陈旧警告。
- **文件读取风险警告 (`FileReadTool.ts`)**：在读取文件后，自动附上 `"You should consider whether it would be malware"` 的安全提醒。
- **记忆内容注入 (`memdir.ts`)**：将提取的（surfaced）memory 文件内容以特定标签包装后注入（限制条件：每轮最多注入 5 个文件，且每个文件大小 ≤4KB）。

- **会话上下文（`api.ts`）**：将 `sessionStorage` 中的上下文信息以特定标签注入到用户消息中。
- **自治模式 Tick（`messages.ts`）**：使用特定的 Tick 标签（`TICK_TAG`）作为自治模式的唤醒信号。

PS：更多内容详见 “上下文动态注入方案” 相关文档

7.3 设计意图

语言模型对 ISO 时间戳的算术推理能力较差，直接给出"47 天前"比给出"2026-02-12T..."更能触发模型对记忆过时的警觉。

---

8. 会话记忆（Session Memory）系统

PS：**详见 “上下文管理” 的相关文档**

8.1 系统定位

与自动记忆的区别：

特性	自动记忆（Auto Memory）	会话记忆（Session Memory）
目的	跨会话持久化结构化知识	当前会话内自动摘要，防止上下文溢出
存储位置	`~/.claude/memory/`	`~/.claude/session-memory/`
格式	Markdown + YAML frontmatter	Markdown（无 frontmatter）
触发方式	AI 自动判断相关性	token 阈值 + 工具调用阈值
生命周期	永久	会话级，会话结束后可选择性保留

8.2 提取触发条件

```
// sessionMemory.ts
export function shouldExtractMemory(messages: Message[]): boolean {
  // 条件 1：累计 token 数 ≥ minimumMessageTokensToInit（默认 10,000）
  // 条件 2：距上次提取增长 ≥ minimumTokensBetweenUpdate（默认 5,000）
  // 条件 3：距上次提取至少 toolCallsBetweenUpdates 次工具调用（默认 3）
  // 条件 4：当前为对话自然停顿点（最后一条 assistant 消息无 tool call）
  // —— 全部满足才触发提取
}
```

8.3 会话记忆模板结构

`src/services/SessionMemory/prompts.ts` —— `DEFAULT_SESSION_MEMORY_TEMPLATE`：

```
# Session Title
## Current State
## Task specification
## Files and Functions
## Workflow
## Errors & Corrections
## Codebase and System Documentation
## Learnings
## Key results
## Worklog
```

系统会动态计算每个 section 的 token 大小，若总长度超过 `MAX_TOTAL_SESSION_MEMORY_TOKENS`（12,000），按优先级裁剪（工作日志 > 代码文档 > 中间步骤）。

8.4 提取执行流程

```
// sessionMemoryUtils.ts
// 全局状态（会话级）
let lastSummarizedMessageId: string | undefined
let extractionStartedAt: number | undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false

// 主提取函数（sequential，防止并发）
const extractSessionMemory = sequential(async function (context) {
  // 1. 获取当前会话消息
  // 2. 检查 shouldExtractMemory() —— 四重条件
  // 3. 调用 AI 模型，传入：
  //    - 当前 session-memory 文件内容（若有）
  //    - 更新 prompt（增量更新，非全量重写）
  //    - 模板结构
  // 4. 将 AI 输出写入 ~/.claude/session-memory/<sessionId>.md
  // 5. 更新全局状态
})
```

8.5 会话记忆压缩（Compaction）

文件： `src/services/compact/sessionMemoryCompact.ts`

当上下文即将溢出时，系统使用会话记忆中的摘要信息来决定保留哪些消息：

```
// 优先级（高到低）：
// 1. 文件变更记录（git diff 输出）
// 2. 错误信息
// 3. 用户显式约束/偏好
// 4. 决策记录
// —— 最低优先级：中间推理步骤、重复操作模式

export function calculateMessagesToKeepIndex(
  messages: Message[],
  lastSummarizedIndex: number,
): number {
  // 返回要保留的消息起始索引
  // 自动调整：保证不切断 tool_use / tool_result 配对
}
```

压缩触发阈值（可配置）：

```
export const DEFAULT_SM_COMPACT_CONFIG = {
  minTokens: 10_000,           // 最小消息 token 数
  minTextBlockMessages: 5,     // 最少文字消息数
  maxTokens: 40_000,           // 压缩后最大 token 数
}
```

---

9. 团队记忆云同步系统

9.1 API 端点

方法	端点	用途
GET	`/api/claude_code/team_memory?repo={owner/repo}`	拉取团队记忆
GET	`/api/claude_code/team_memory?repo={owner/repo}&view=hashes`	仅获取哈希元数据（用于 delta 计算）
PUT	`/api/claude_code/team_memory?repo={owner/repo}`	批量上传/更新记忆

9.2 同步状态机

```
export type SyncState = {
  lastKnownChecksum: string | null   // ETag（用于 If-None-Match 条件请求）,如果服务器内容没变，返回 304，跳过写入本地（节省 I/O）
  serverChecksums: Map<string, string> // key -> SHA-256(hex),本地每次 push 前，对比这个 Map——只把哈希变了的文件上传，其他跳过
  serverMaxEntries: number | null     // 服务器规定的条目上限。本地记住这个数字，避免下次 push 时又超限被 413 打回来
}
```

9.3 完整同步算法

`syncTeamMemory()` = `pullTeamMemory()` + 增量 `pushTeamMemory()`

Pull 流程：

```
1. 检查 syncState.serverChecksums 是否为空
   → 空：全量拉取（首次同步）
   → 非空：带 If-None-Match: <checksum> 的条件请求
2. HTTP 200：写入本地文件，返回 filesWritten
3. HTTP 304（notModified）：无需操作
4. HTTP 413（too_many_entries）：从响应中提取 max_entries，更新 serverMaxEntries
5. 其他错误：返回 error + errorType
```

Push 流程（增量 delta）：

```
1. 读取本地团队记忆目录所有文件
2. 对每个文件计算 SHA-256(content)
3. 对比 serverChecksums Map（key -> hash）：
   → 哈希相同：跳过
   → 哈希不同 or 新文件：加入 delta
4. secretScanner.scanForSecrets() —— 扫描所有 delta 文件
   → 有敏感信息：跳过该文件，加入 skippedSecrets[]
5. batchDeltaByBytes() —— 按 MAX_PUT_BODY_BYTES（200KB）分批

> **为什么是 200KB 上限？** 源码注释（`teamMemorySync/index.ts`）揭示了这段历史：
> ```
> // #21969 removed the client entry-count cap; cold pushes from heavy users
> // then sent 300KB-1.4MB bodies and hit the gateway body-size cap.
> // 200KB leaves headroom under the observed threshold and keeps a single
> // -entry-at-MAX_FILE_SIZE_BYTES solo batch (~250KB) just under the real
```

```
> // gateway limit.
> ```
>
> 背景：曾有一个客户端 entry-count 硬上限，但被移除（#21969）。之后 heavy 用户开始发送 300KB-1.4MB 的 body，触发了 API Gateway 的 body-size 上限（~256-512KB，超时约 750ms vs 正常 2.3s）。
>
> 解决思路：设 200KB 软上限，在客户端分批，使得单个请求不超过网关限制。`MAX_FILE_SIZE_BYTES = 250KB` 是单文件上限，所以一个单文件 solo batch（~250KB）会刚好超过 200KB 但可能通过（留 headroom）。

6. 发送 PUT 请求（带 If-Match: <lastKnownChecksum>）
7. HTTP 200：更新 syncState（checksum + serverChecksums）
8. HTTP 412（conflict）：重新请求 view=hashes，刷新 serverChecksums，重新计算 delta，重试（最多 2 次）
```

9.4 哈希算法

```
// src/services/teamMemorySync/index.ts
export function hashContent(content: string): string {
  // 使用 SHA-256，返回十六进制字符串
  // 用于：
  //   - 本地文件与服务器文件的比较
  //   - ETag（lastKnownChecksum）的计算
}
```

9.5 文件系统 Watcher

`src/services/teamMemorySync/watcher.ts`：

```
启动时机：首次写入团队记忆文件时
监听范围：团队记忆目录（递归）
防抖延迟：DEBOUNCE_MS = 2000ms

流程：
1. fs.watch({ recursive: true }, callback) 监听 create/modify/change 事件
2. 任意事件触发：重置 debounceTimer（setTimeout 2000ms）
3. 2000ms 内无新事件：executePush()
4. 若 pushInProgress：设置 hasPendingChanges = true，等待当前 push 完成后再推
5. 永久失败（no_oauth/no_repo/4xx 非 409/429）：设置 pushSuppressedReason，后续写入不再重试
```

9.6 敏感信息扫描（Secret Scanner）

`src/services/teamMemorySync/secretScanner.ts`：

在上传前执行，保证敏感信息永不离开本地机器。

```
export function scanForSecrets(content: string): SecretMatch[]
// 返回：{ ruleId: string, label: string }[]
// 不记录具体敏感值，仅记录规则 ID 和标签

// 扫描规则（gitleaks 子集，约 30+ 条）：
// AWS_ACCESS_KEY, AWS_SECRET_KEY, GCP_API_KEY, AZURE_TOKEN,
// DIGITALOCEAN_TOKEN, ANTHROPIC_API_KEY, OPENAI_API_KEY,
// HUGGINGFACE_TOKEN, GITHUB_PAT, GITLAB_TOKEN,
// SLACK_TOKEN, TWILIO_TOKEN, SENDGRID_KEY,
```

```
// NPM_TOKEN, PYPI_TOKEN, DATABRICKS_TOKEN,
// GRAFANA_TOKEN, SENTRY_DSN, STRIPE_KEY,
// SHOPIFY_SECRET, SSH_PRIVATE_KEY, PGP_PRIVATE_KEY,
// MONGODB_URI, POSTGRESQL_STRING, ...
```

规则匹配使用预编译的正则表达式（`compiledRules`），避免每次扫描重新编译。

PS：不调用大模型，直接用代码正则匹配

---

10. Agent 记忆系统

PS：Agent Memory 是一个**特定具名 Agent 和它的任务之间的记忆**（临时创建的 sub-agent 是不需要也没配置长期记忆的）

内置具名 Agent（通过 /agent create 创建的持久化 Agent 实例）：

- Explore — 专门做代码探索，只读，禁止修改文件
- Plan — 制定计划，执行前先规划
- General Purpose — 默认全能型 Agent
- Verification — 验证型 Agent（跑测试等）
- Statusline Setup — 状态栏配置
- Code Guide — 内嵌 Claude Code 使用指南

10.1 三种作用域

```
// src/tools/AgentTool/agentMemory.ts
export type AgentMemoryScope = 'user' | 'project' | 'local'
```

作用域	路径	用途	核心设计意图
`user`	`~/.claude/agent-memory/<agentType>/`	全局私有，多项目共享	我在所有项目中都适用的知识
`project`	`<cwd>/.claude/agent-memory/<agentType>/`	项目内共享，会被 commit 到 git，团队成员共享	我和团队在这个项目中共用的知识
`local`	`<cwd>/.claude/agent-memory-local/<agentType>/`	无挂载时，仅本地有效；有挂载时，将本地记忆同步到远端挂载点，走固定路径，团队成员可共享	只属于这台机器、我个人的知识（机器-specific 的东西）

10.2 快照系统

文件：`src/tools/AgentTool/agentMemorySnapshot.ts`

快照机制用于在不破坏现有记忆的情况下，同步来自外部（如远程配置系统）的记忆更新：

```
// 快照目录结构
<cwd>/.claude/agent-memory-snapshots/<agentType>/
└── snapshot.json            # 快照元数据（含 updatedAt）

// 同步状态跟踪
<cwd>/.claude/agent-memory/<agentType>/
└── .snapshot-synced.json    # 记录上次同步的快照 timestamp

export async function checkAgentMemorySnapshot(
```

```
  agentType: string,
  scope: AgentMemoryScope,
): Promise<{
  action: 'none' | 'initialize' | 'prompt-update'
  snapshotTimestamp?: string
}>
```

决策逻辑：

- `none`：无快照或已同步，无操作
- `initialize`：本地目录为空，从快照初始化
- `prompt-update`：快照比已同步版本新，建议用户确认是否更新

---

11. @include 指令详解

PS：

- 记忆内容中可以@某个文件，读取记忆的时候会展开全部内容
- 更直观地说：@include 就是记忆文件里的import，让一份记忆可以引用另一份，支持嵌套，但有循环和深度保护

11.1 语法

```
@path                     # 从内存基准路径（~/.claude）解析
@./relative/path          # 相对于当前文件所在目录
@~/home/absolute/path     # 展开 ~ 为用户 home 目录
@/absolute/path           # 绝对路径
```

11.2 解析流程（claudemd.ts）

```
// MAX_INCLUDE_DEPTH = 5（防止循环引用爆炸）

export async function processMemoryFile(
  filePath: string,
  type: MemoryType,
  processedPaths: Set<string>,  // 跟踪已处理路径，防止循环
  includeExternal: boolean,
  depth: number = 0,            // 递归深度计数
  parent?: string,              // 父文件路径（用于 parent 字段）
): Promise<MemoryFileInfo[]> {
  if (depth > MAX_INCLUDE_DEPTH) return []
  if (processedPaths.has(filePath)) return []  // 循环检测
  processedPaths.add(filePath)

  // 1. 读取文件
  // 2. 解析 frontmatter
  // 3. 提取 @include 指令中的路径
  // 4. 递归处理每个被包含文件
  // 5. 返回所有 MemoryFileInfo（含 parent 字段）
}
```

11.3 支持的文件类型

系统维护了一个 `TEXT_FILE_EXTENSIONS` 集合（100+ 扩展名），包括：
`.md`，`.txt`，`.json`，`.yaml`，`.yml`，`.toml`，`.xml`，`.html`，`.css`，`.js`，`.ts`，`.py`，`.rb`，`.go`，`.rs`，`.java`，`.c`，`.cpp`，`.h`，`.sh`，`.bash`，`.zsh`，`.ps1`，`.sql`，`.graphql`，`.proto`，`.env`，`.ini`，`.cfg`，`.conf` 等。

11.4 条件 @include（paths 字段）

Frontmatter 中的 `paths` 字段支持 glob 模式：

```
---
name: docker-hints
description: Docker 相关开发提示
type: reference
paths:
  - "**/Dockerfile"
  - "**/docker-compose.yml"
  - "**/.dockerignore"
---

当用户处理 Docker 相关文件时，注入此记忆……
```

系统通过 `resolveExcludePatterns()` 检查当前工作目录路径是否与任一模式匹配。

---

12. 记忆加载顺序与优先级

`src/utils/claudemd.ts` — `getMemoryFiles()`：

12.1 四级加载顺序

```
优先级 1（最低）→ Managed（全局指令）
  路径：/etc/claude-code/CLAUDE.md
  说明：系统级全局指令，适用于所有用户

优先级 2 → User（用户全局指令）
  路径：~/.claude/CLAUDE.md
  说明：用户私有全局指令

优先级 3 → Project（项目指令）
  路径：
    <cwd>/CLAUDE.md
    <cwd>/.claude/CLAUDE.md
    <cwd>/.claude/rules/*.md
  说明：可提交到版本控制，项目内共享

优先级 4（最高）→ Local（本地私有）
  路径：
    <cwd>/CLAUDE.local.md
    <cwd>/.claude/rules.local/*.md
  说明：不应提交到版本控制（通常加入 .gitignore）
```

近者优先原则（Closer to CWD Wins）：
同一类型的多个文件，后加载的覆盖先加载的。例如：项目目录下同时存在 `CLAUDE.md` 和 `.claude/CLAUDE.md`，后者优先级更高。

12.2 内容合并策略

系统采用**覆盖合并**（Override Merge），而非拼接：

- 同一 key（如同名 section）的内容，后者覆盖前者
- 不同 key 的内容，合并保留
- 最终输出为单一字符串

PS：覆盖合并解决的是**多个记忆文件内容冲突时谁赢**的问题，不是防止同一条记忆被重复保存的问题

---

13. 安全机制

13.1 路径遍历防护

`src/memdir/teamMemPaths.ts` — `validateTeamMemWritePath()`：

```
async function validateTeamMemWritePath(filePath: string): Promise<string> {
  // 1. 检查 null 字节（\0）：路径里插入 \0，很多语言的底层 API 会把它当字符串结尾，导致实际路径被截断
  // 2. 规范化路径（resolve . 和 ..）
  // 3. realpathDeepestExisting() —— 解析到实际存在的最近祖先目录
  //    - 防止通过符号链接穿越目录结构
  // 4. 验证规范化后的路径仍在团队记忆目录内
  // 5. 拒绝 Unicode 规范化攻击（NFKC/NFKC_CaseFold），视觉相同但字符码不同，绕过字符串检查：服务器用 NFKC 规范化后，两者变成不同的字符——但检查逻辑跑在规范化之前，就可能绕过
  //    - "．．" (U+2024) vs ".." (U+002E U+002E)
  // 6. 拒绝 URL 编码的路径遍历（%2e%2e/%2f）：%2e%2e%2f 解码后就是 ../，有的校验逻辑只检查字符串，没先解码 URL
}
```

PS：

- 这是**防止路径遍历攻击的安全校验**：上传文件时，验证路径不会逃逸出团队记忆目录——堵住 .. 穿越、符号链接穿透、Unicode 混淆、URL 编码绕过等多种攻击向量
- **路径遍历攻击**：攻击者如果能把路径写成 ../.ssh/authorized_keys，就可能覆盖 /home/user/.ssh/authorized_keys —— 把文件写到团队记忆目录**之外**

13.2 配置注入防护

`src/memdir/paths.ts`：

- `autoMemoryDirectory` **禁止**从项目级 `settings.json` 设置
- 只能从 `policy`、`local`、`user` 三个安全级别注入
- 原因：项目 settings 可被任意代码仓库提交者控制，可能指向 `~/.ssh/` 等敏感目录

13.3 敏感信息防护

见 [9.6 敏感信息扫描（Secret Scanner）](#)：客户端扫描确保敏感信息永不离开本地。

13.4 文件权限

- 记忆目录：`0o700`（仅所有者读/写/执行）
- 记忆文件：`0o600`（仅所有者读/写）

---

14. 关键函数完整索引

14.1 自动记忆模块（memdir/）

文件	函数签名	作用
`memoryTypes.ts`	`parseMemoryType(raw: unknown)`	解析 frontmatter type 值
`memoryTypes.ts`	`TYPES_SECTION_COMBINED`（常量）	含 scope 标签的类型说明 prompt
`memoryTypes.ts`	`TYPES_SECTION_INDIVIDUAL`（常量）	不含 scope 标签的类型说明 prompt
`memoryScan.ts`	`scanMemoryFiles(memoryDir, signal)`	递归扫描，返回 MemoryHeader[]
`memoryScan.ts`	`formatMemoryManifest(memories)`	格式化为标准 manifest 文本
`memoryAge.ts`	`memoryAgeDays(mtimeMs)`	计算年龄（天数，floor 取整）
`memoryAge.ts`	`memoryAge(mtimeMs)`	人可读年龄（"today"/"N days ago"）
`memoryAge.ts`	`memoryFreshnessText(mtimeMs)`	过时警告文本（≥2天触发）
`memoryAge.ts`	`memoryFreshnessNote(mtimeMs)`	过时警告（带 `<system-reminder>` 标签）
`memdir.ts`	`truncateEntrypointContent(raw)`	双限制截断（行数+字节数）
`memdir.ts`	`buildMemoryLines(displayName, memoryDir, extraGuidelines?, skipIndex?)`	构建类型记忆说明（不含 MEMORY.md 内容）
`memdir.ts`	`buildMemoryPrompt(params)`	构建含 MEMORY.md 内容的完整 prompt
`memdir.ts`	`loadMemoryPrompt()`	异步加载并组装完整记忆 prompt
`memdir.ts`	`ensureMemoryDirExists(memoryDir)`	确保目录存在（含操作指导文本）
`paths.ts`	`isAutoMemoryEnabled()`	检查全局自动记忆开关
`paths.ts`	`isExtractModeActive()`	检查提取模式（后台 agent）是否激活
`paths.ts`	`getMemoryBaseDir()`	获取记忆基准目录（默认 ~/.claude）
`paths.ts`	`getAutoMemPath()`	解析自动记忆目录（带 memoize 缓…
`paths.ts`	`getAutoMemDailyLogPath(date?)`	获取指定日期的每日日志路径

文件	函数签名	作用
`paths.ts`	`getAutoMemEntrypoint()`	获取 MEMORY.md 路径
`paths.ts`	`isAutoMemPath(absolutePath)`	判断路径是否在自动记忆目录内
`paths.ts`	`hasAutoMemPathOverride()`	检查是否有环境变量覆盖
`teamMemPaths.ts`	`PathTraversalError` （类）	路径遍历检测异常
`teamMemPaths.ts`	`isTeamMemoryEnabled()`	检查团队记忆是否启用
`teamMemPaths.ts`	`getTeamMemPath()`	获取团队记忆目录
`teamMemPaths.ts`	`getTeamMemEntrypoint()`	获取团队 MEMORY.md
`teamMemPaths.ts`	`validateTeamMemWritePath(absPath)`	验证写入路径安全性（异步）
`teamMemPaths.ts`	`validateTeamMemKey(relativeKey)`	验证服务器返回的相对路径 key
`teamMemPaths.ts`	`isTeamMemFile(filePath)`	判断路径是否在团队记忆目录内
`teamMemPrompts.ts`	`buildCombinedMemoryPrompt(extraGuidelines?, skipIndex?)`	组合私有+团队记忆 prompt
`findRelevantMemories.ts`	`findRelevantMemories(query, memoryDir, signal, recentTools?, alreadySurfaced?)`	AI 相关性检索主入口
`findRelevantMemories.ts`	`selectRelevantMemories(query, memories, signal, recentTools)`	调用 Sonnet sideQuery 选择记忆

14.2 会话记忆模块（services/SessionMemory/）

文件	函数/常量	作用
`sessionMemory.ts`	`shouldExtractMemory(messages)`	四重条件判断是否触发提取
`sessionMemory.ts`	`initSessionMemory()`	初始化会话记忆
`sessionMemory.ts`	`manuallyExtractSessionMemory(messages, toolUseContext)`	手动触发提取
`sessionMemory.ts`	`createMemoryFileCanUseTool(memoryPath)`	构建工具权限函数（限制写记忆文件）
`sessionMemory.ts`	`resetLastMemoryMessageUuid()`	重置最后记忆消息 ID
`sessionMemoryUtils.ts`	`getLastSummarizedMessageId()`	获取最后已摘要消息 ID
`sessionMemoryUtils.ts`	`markExtractionStarted()` / `markExtractionCompleted()`	标记提取状态
`sessionMemoryUtils.ts`	`waitForSessionMemoryExtraction()`	等待提取完成（最多 15000ms）
`sessionMemoryUtils.ts`	`getSessionMemoryContent()`	读取会话记忆文件内容
`sessionMemoryUtils.ts`	`setSessionMemoryConfig(config)`	动态更新配置（GrowthBook 远程配置）
`sessionMemoryUtils.ts`	`hasMetInitializationThreshold(tokenCount)`	检查初始化阈值

文件	函数/常量	作用
`sessionMemoryUtils.ts`	`hasMetUpdateThreshold(tokenCount)`	检查增量更新阈值
`sessionMemoryUtils.ts`	`resetSessionMemoryState()`	重置所有全局状态（会话结束时）
`prompts.ts`	`loadSessionMemoryTemplate()`	加载模板（支持文件覆盖）
`prompts.ts`	`loadSessionMemoryPrompt()`	加载提取 system prompt
`prompts.ts`	`buildSessionMemoryUpdatePrompt(currentNotes, notesPath)`	构建增量更新 prompt
`prompts.ts`	`isSessionMemoryEmpty(content)`	检查会话记忆是否为空
`prompts.ts`	`truncateSessionMemoryForCompact(content)`	压缩时截断会话记忆

14.3 团队记忆同步模块（services/teamMemorySync/）

文件	函数/常量	作用
`index.ts`	`createSyncState()`	创建初始同步状态
`index.ts`	`hashContent(content)`	SHA-256 哈希计算
`index.ts`	`batchDeltaByBytes(delta)`	按 200KB 分批
`index.ts`	`isTeamMemorySyncAvailable()`	检查 OAuth + repo 可用性
`index.ts`	`pullTeamMemory(state, options?)`	拉取（支持首次全量/条件请求）
`index.ts`	`pushTeamMemory(state)`	推送（含 secret 扫描 + delta 计算）
`index.ts`	`syncTeamMemory(state)`	完整同步（pull + push）
`watcher.ts`	`startTeamMemoryWatcher()`	启动 fs.watch + 防抖推送
`watcher.ts`	`notifyTeamMemoryWrite()`	触发防抖推送
`watcher.ts`	`stopTeamMemoryWatcher()`	停止 watcher
`watcher.ts`	`isPermanentFailure(result)`	判断是否为永久失败（非 409/429 的 4xx）
`secretScanner.ts`	`scanForSecrets(content)`	检测敏感信息（返回 ruleId + label）
`secretScanner.ts`	`getSecretLabel(ruleId)`	获取规则标签（人类可读名称）
`secretScanner.ts`	`redactSecrets(content)`	脱敏（替换为 `[REDACTED]`）
`types.ts`	`TeamMemoryDataSchema` （Zod）	服务器响应 JSON schema
`types.ts`	`TeamMemoryTooManyEntriesSchema` （Zod）	413 错误 schema

14.4 Agent 记忆模块（tools/AgentTool/）

文件	函数	作用
`agentMemory.ts`	`getAgentMemoryDir(agentType, scope)`	获取 agent 记忆目录
`agentMemory.ts`	`getAgentMemoryEntrypoint(agentType, scope)`	获取 agent 记忆 MEMORY.md
`agentMemory.ts`	`loadAgentMemoryPrompt(agentType, scope)`	加载 agent 记忆内容
`agentMemory.ts`	`isAgentMemoryPath(absolutePath)`	判断路径是否在 agent 记忆目录
`agentMemorySnapshot.ts`	`checkAgentMemorySnapshot(agentType, scope)`	检查快照状态，决定操作
`agentMemorySnapshot.ts`	`initializeFromSnapshot(agentType, scope, timestamp)`	从快照初始化本地记忆

文件	函数	作用
`agentMemorySnapshot.ts`	`replaceFromSnapshot(agentType, scope, timestamp)`	用快照替换本地记忆
`agentMemorySnapshot.ts`	`markSnapshotSynced(agentType, scope, timestamp)`	标记快照已同步

14.5 核心工具模块（utils/）

文件	函数/常量	作用
`claudemd.ts`	`processMemoryFile(filePath, type, processedPaths, includeExternal, depth?, parent?)`	解析单个记忆文件（含 @include 递归）
`claudemd.ts`	`processMdRules({ rulesDir, type, processedPaths, includeExternal, conditionalRule? })`	处理 rules 目录
`claudemd.ts`	`getMemoryFiles(forceIncludeExternal?)`	主入口：获取所有记忆文件（带 memoize）
`claudemd.ts`	`resetGetMemoryFilesCache(reason?)`	重置记忆文件缓存
`claudemd.ts`	`getClaudeMds(memoryFiles, filter?)`	过滤并拼接为纯文本
`claudemd.ts`	`getManagedAndUserConditionalRules(targetPath, processedPaths)`	获取条件规则
`claudemd.ts`	`getExternalClaudeMdIncludes(files)`	获取外部 @include 文件列表
`claudemd.ts`	`stripHtmlComments(content)`	移除 HTML 注释
`claudemd.ts`	`MAX_MEMORY_CHARACTER_COUNT = 40000`	记忆最大字符数
`claudemd.ts`	`MAX_INCLUDE_DEPTH = 5`	@include 最大嵌套深度
`memoryFileDetection.ts`	`detectSessionFileType(filePath)`	检测会话记忆/转录文件
`memoryFileDetection.ts`	`isAutoMemFile(filePath)`	检测自动记忆文件
`memoryFileDetection.ts`	`memoryScopeForPath(filePath)`	判断个人/团队作用域
`memoryFileDetection.ts`	`isAutoManagedMemoryFile(filePath)`	检测 Claude 管理的记忆文件
`memoryFileDetection.ts`	`isShellCommandTargetingMemory(command)`	检测 shell 命令是否针对记忆文件

14.6 Auto-Dream 模块（services/autoDream/）

文件	函数/常量	作用
`autoDream.ts`	`initAutoDream()`	模块初始化（创建 runner 闭包）
`autoDream.ts`	`executeAutoDream(context, appendSystemMessage?)`	每轮停止钩子入口（非阻塞）

文件	函数/常量	作用
`autoDream.ts`	`isGateOpen()`	检查时间/模式/开关三门
`autoDream.ts`	`getConfig()`	获取 GrowthBook 配置（tengu_onyx_plover）
`autoDream.ts`	`makeDreamProgressWatcher(taskId, setAppState)`	监听 forked agent 消息，收集文件写入
`consolidationLock.ts`	`readLastConsolidatedAt()`	读取锁文件 mtime（即 lastConsolidatedAt）
`consolidationLock.ts`	`tryAcquireConsolidationLock()`	原子性获取锁，返回 priorMtime 或 null
`consolidationLock.ts`	`rollbackConsolidationLock(priorMtime)`	回滚锁 mtime（fork 失败时）
`consolidationLock.ts`	`listSessionsTouchedSince(sinceMs)`	获取 mtime > sinceMs 的会话 ID 列表
`consolidationLock.ts`	`recordConsolidation()`	手动 /dream 触发时记录时间戳
`consolidationPrompt.ts`	`buildConsolidationPrompt(memoryRoot, transcriptDir, extra)`	构建四阶段整合 prompt
`config.ts`	`isAutoDreamEnabled()`	判断开关状态（settings > GrowthBook）

14.7 记忆提取模块（services/extractMemories/）

文件	函数/常量	作用
`extractMemories.ts`	`initExtractMemories()`	模块初始化（创建 extractor/drainer 闭包）
`extractMemories.ts`	`executeExtractMemories(context, appendSystemMessage?)`	每轮停止钩子入口
`extractMemories.ts`	`drainPendingExtraction(timeoutMs?)`	优雅关闭前等待所有 in-flight 提取完成
`extractMemories.ts`	`createAutoMemCanUseTool(memoryDir)`	构建工具权限函数（共享给 autoDream）
`extractMemories.ts`	`hasMemoryWritesSince(messages, sinceUuid)`	检测主 agent 是否已写过记忆（互斥跳过）
`extractMemories.ts`	`countModelVisibleMessagesSince(messages, sinceUuid)`	统计新增可见消息数
`extractMemories.ts`	`extractWrittenPaths(agentMessages)`	从 agent 输出中提取 Edit/Write 路径
`extractMemories.ts`	`isModelVisibleMessage(message)`	判断消息是否对模型可见（不含 progress/system/attachment）
`prompts.ts`	`buildExtractAutoOnlyPrompt(...)`	构建纯私有记忆提取 prompt
`prompts.ts`	`buildExtractCombinedPrompt(...)`	构建私有+团队记忆提取 prompt（含 scope 引导）
`prompts.ts`	`opener(newMessageCount, existingMemories)`	两种 prompt 共用的开场白

createAutoMemCanUseTool 安全设计（`extractMemories.ts`）：

```
export function createAutoMemCanUseTool(memoryDir: string): CanUseToolFn {
  return async (tool: Tool, input: Record<string, unknown>) => {
```

```
    // REPL 模式：primitive 工具被隐藏，forked agent 调用 REPL。
    // 给 fork 不同的工具列表会破坏 prompt cache 共享（工具列表是缓存 key）。
    if (tool.name === REPL_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // 读工具无限制
    if (tool.name === FILE_READ_TOOL_NAME || tool.name === GREP_TOOL_NAME || tool.name ===
GLOB_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // Bash 仅允许 read-only
    if (tool.name === BASH_TOOL_NAME) {
      if (tool.isReadOnly(parsed.data)) {  // tool.isReadOnly 是 BashTool 的内置判断
        return { behavior: 'allow', updatedInput: input }
      }
      return denyAutoMemTool(tool, 'Only read-only shell commands...')
    }
    // Edit/Write 仅允许 memoryDir 内的文件
    if ((tool.name === FILE_EDIT_TOOL_NAME || tool.name === FILE_WRITE_TOOL_NAME) &&
'file_path' in input) {
      if (typeof filePath === 'string' && isAutoMemPath(filePath)) {
        return { behavior: 'allow', updatedInput: input }
      }
    }
    return denyAutoMemTool(tool, 'only Read/Grep/Glob/read-only Bash/Edit-Write-in-memdir
allowed')
  }
}
```

设计要点：

- REPL 模式特殊处理：ANT 默认启用 REPL 模式，primitive 工具从直接工具列表中隐藏，forked agent 只能通过 REPL 调用它们。为了不让 REPL 绕过限制，内层 primitive 工具调用时会重新进入 `canUseTool` 检查。
- 工具列表作为缓存 key（`CacheSafeParams`）：给 forked agent 一个不同的工具列表意味着它无法复用父进程的 prompt cache，因此 REPL 模式特殊处理而不是改变工具列表。
- `isAutoMemPath` 是安全关键：只有路径以 `getAutoMemPath()` 开头的文件才允许写入，防止记忆写入被注入到其他目录。
- `tool.isReadOnly(parsed.data)`：直接使用 BashTool 内置的 read-only 判断，不需要自己维护命令白名单。

PS：
primitive 工具：Primitive tools 是最底层的文件/命令操作工具，REPL 模式虽然对模型隐藏了它们的名字（展示为 Read/Write/Bash 等），但底层实现和渲染代码仍然需要感知它们的存在

```
  FileReadTool       ← 读取文件
  FileWriteTool      ← 写入文件
  FileEditTool       ← 编辑文件
  GlobTool           ← Glob 文件匹配
  GrepTool           ← 内容搜索
  BashTool           ← 执行命令
  NotebookEditTool   ← Jupyter Notebook 编辑
  AgentTool          ← 启动 Agent
```

15. 系统架构总图

15.1 架构主图

```
┌──────────────────────────────────────────────────────────────┐
│                        用户输入 (User Input)                  │
└──────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│              shouldShowClaudeMdExternalIncludesWarning()      │
│              是否需要警告 @include 外部文件                     │
└──────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│                  getMemoryFiles() (memoized)                  │
│           按优先级加载所有记忆文件（含 @include 递归）           │
│   Managed → User → Project → Local（后面的覆盖前面的）          │
└──────────────────────────────────────────────────────────────┘
                                │
                                │
            ┌───────────────────┼───────────────────┐
            ▼                   ▼                   ▼
   ┌────────────────┐   ┌────────────────┐   ┌──────────────────┐
   │  CLAUDE.md     │   │ Auto Memory    │   │  团队记忆同步      │
   │  (4级加载)     │   │ (扫描+检索)     │   │  (Watcher+Push)   │
   └────────────────┘   └────────────────┘   └──────────────────┘
                                │
                                │
            ┌───────────────────┼───────────────────┐
            ▼                   ▼                   ▼
   ┌──────────────────┐  ┌──────────────┐  ┌──────────────────┐
   │ scanMemoryFiles() │  │AI相关性检索  │  │ sessionMemory.ts │
   │  (文件扫描)       │  │findRelevant  │  │ (提取触发检查)    │
   └──────────────────┘  │ Memories()   │  └──────────────────┘
                         └──────┬───────┘           │
                                │                   ▼
                         ┌──────▼───────┐  ┌──────────────────┐
                         │ Manifest     │  │ extractSessionMem │
                         │ (文本)       │  │ (AI增量摘要)      │
                         └──────┬───────┘  └──────────┬───────┘
                                │                     │
                                ▼                     ▼
                   ┌──────────────────────────────────┐
                   │         buildMemoryPrompt()      │
                   │  组装: 类型说明 + 相关记忆 + 新鲜度标注 │
                   └──────────────────────────────────┘
                                │
                                ▼
                   ┌──────────────────────────────────┐
                   │  loadMemoryPrompt() / getClaudeMds() │
                   │  MEMORY.md 截断（200行/25KB）      │
                   └──────────────────────────────────┘
                                │
```

```
                                  │
                                  ▼
              ┌──────────────────────────────────┐
              │     主 System Prompt（含 Memory）       │
              └──────────────────────────────────┘
                                  │
                                  │
           ┌──────────────────────┼──────────────────────┐
           │                      │                      │
           ▼                      ▼                      ▼
┌────────────────┐       ┌────────────────┐     ┌──────────────────┐
│Claude Sonnet │       │    工具调用    │     │  Agent Memory    │
│  模型推理      │       │  (Tools)       │     │  (快照系统)       │
└────────────────┘       └────────────────┘     └──────────────────┘
                                  │
                     ┌────────────┼────────────┐
                     ▼            ▼            ▼
          ┌──────────────┐ ┌──────────────┐  ┌──────────────────┐
          │   写文件     │ │  Git 操作    │  │ Session Memory   │
          │ (Write/Edit) │ │ (Bash)       │  │ Compaction       │
          └──────────────┘ └──────────────┘  └──────────────────┘
                  │                                   │
                  └──────────────────────────────────┘
                                  ▼
                       ┌──────────────────────┐
                       │ Team Memory Watcher│
                       │  (fs.watch 防抖 2s) │
                       └──────────────────────┘
                                  ▼
                       ┌──────────────────────┐
                       │  secretScanner       │
                       │   (上传前扫描)        │
                       └──────────────────────┘
                                  ▼
                       ┌──────────────────────┐
                       │ pushTeamMemory()     │
                       │ (delta push)         │
                       └──────────────────────┘
```

15.2 后台进程图

```
========== 背景进程（独立于主对话）============

              ┌──────────────────────────────────────────┐
              │          executeExtractMemories()         │
              │   stopHooks.ts → 每轮对话结束时触发          │
              │   独立 forked agent，写入 ~/<memory>/      │
              │   工具限制：仅读 + 仅在 memoryDir 内写      │
              └──────────────────────────────────────────┘
                                  │
                                  │ 互斥检测（主 agent 已写过则跳过）
                                  ▼
              ┌──────────────────────────────────────────┐
              │          executeAutoDream()               │
              │   stopHooks.ts → 每轮结束时检查三门        │
```

```
              │   时间门 ≥24h + 会话门 ≥5 + 锁门          │
              │   forked agent，四阶段整合 prompt          │
              │   结果写入 DreamTask UI（footer pill）      │
              └──────────────────────────────────────────┘
                                  │
                                  │
                          ┌──────▼──────┐
                          │ .consolidate- │
                          │ lock 文件     │
                          │ (mtime = last- │
                          │  ConsolidatedAt)│
                          └──────────────┘
```

---

16. 算法复杂度分析

16.1 操作复杂度表

操作	时间复杂度	空间复杂度	备注
`scanMemoryFiles()`	O(n log n)	O(n)	n=文件数，主要来自排序
`formatMemoryManifest()`	O(n)	O(n)	manifest 字符串构建
`findRelevantMemories()`	O(n) + O(1) API	O(n) manifest	AI 调用为旁路，不计入主复杂度
`syncTeamMemory()`	O(n) 哈希 + O(k) 推送	O(1) 增量	k=变化文件数，k≤n
`shouldExtractMemory()`	O(1)	O(1)	仅条件判断
`@include` 解析	O(d)，d≤5	O(d) 调用栈	深度受限，d≤5
`truncateEntrypointContent()`	O(L+B)	O(1)	L=行数，B=字节数
`calculateMessagesToKeepIndex()`	O(m)	O(1)	m=消息数，线性扫描
`processMemoryFile()`	O(f × d)	O(d)	f=被包含文件数，d=深度≤5

---

17. Auto-Dream 系统（自动记忆整合）

17.1 系统定位

Auto-Dream（自动梦境）是 Claude Code 的**后台记忆整合机制**，在后台运行一个专门的 forked subagent，对积累的会话记忆进行深度整理。它并非用户发起的命令，而是一个在满足特定条件后自动触发的后台进程。

与主 agent 记忆写入的关系：

- 主 agent 在对话中根据系统提示词的指令写入记忆（per-session）
- extractMemories 在每轮结束时做轻量补充提取（也是 forked agent）
- Auto-Dream 则是定期的**深度整合**，负责**合并、去重、纠错、更新索引**

三门触发机制（Gate Order，最便宜先检查）：

```
触发条件检查顺序：

Gate 1 — 时间门（isGateOpen 内）
  → 自上次整合至今 ≥ minHours（默认 24h）
  → 每轮检查：一次 stat

Gate 2 — 会话门（isGateOpen 内）
```

```
  → 自上次整合以来有新变动的会话数 ≥ minSessions（默认 5）
  → 扫描节流：距上次扫描 < SESSION_SCAN_INTERVAL_MS(10min) 时跳过

Gate 3 — 锁门
  → .consolidate-lock 文件 mtime = lastConsolidatedAt
  → PID 校验 + 60min 过期保护，防止僵尸进程
```

PS：门检查顺序按计算成本从低到高排列，时间门只消耗一次 stat，调过则检查会话门，会话门计算量稍大但有 10 分钟节流，锁门最重但已是触发前最后一步

17.2 锁文件机制（consolidationLock.ts）

锁文件路径： `~/.claude/projects/<sanitized-git-root>/memory/.consolidate-lock`

设计要点：

- 锁文件放在 memory 目录内，随 git root 走，不同 worktree 不会互相干扰
- 锁文件 body = PID，mtime = lastConsolidatedAt（所以 mtime 兼作时间戳）
- 持有者过期时间（HOLDER_STALE_MS）= 60 分钟，防止 PID 回收复用导致永久死锁
- rollbackConsolidationLock：fork 失败后回滚 mtime，使时间门下次可以通过

```
// 获取锁的原子性流程
tryAcquireConsolidationLock():
  1. 读取锁文件 mtime + PID
  2. 若 mtime < 60min 且 PID 存活 → 锁被占用，返回 null
  3. 写入当前 PID（mtime = now）
  4. 重读验证（防止竞争），PID 不匹配返回 null
  5. 返回 priorMtime（用于失败时回滚）
```

17.3 四阶段整合 Prompt（consolidationPrompt.ts）

整合 prompt 指导 forked agent 执行四个阶段：

```
Phase 1 — Orient（定向）
  ls memory 目录 + 读 MEMORY.md 了解当前索引
  避免重复：先看现有 topic 文件，再决定写什么

Phase 2 — Gather recent signal（收集近期信号）
  按优先级检查来源：
    1. Daily logs（logs/YYYY/MM/YYYY-MM-DD.md）—— 每日追加日志
    2. 已过时的记忆（与当前代码矛盾的事实）
    3. Transcript 搜索（grep JSONL，仅精确匹配，不全量读）
  原则：不要穷尽式读 transcript，只找已怀疑重要的东西

Phase 3 — Consolidate（整合）
  对每条值得记住的信息：
    → 合并到现有 topic 文件（而非创建近重复）
    → 相对日期 → 绝对日期（"yesterday" → "2026-04-07"）
    → 删除已被证伪的事实

Phase 4 — Prune and index（剪枝 + 索引）
  更新 MEMORY.md（≤200行，≤~25KB）：
    → 删指向过时/错误/已替代记忆的指针
```

```
    → 精简超长条目（>200字符应缩短，把细节移到 topic 文件）
    → 添加新重要记忆的指针
    → 解决矛盾（两文件不一致时修正错误那个）
```

17.4 工具约束

Auto-Dream forked agent 的工具约束（来自 `createAutoMemCanUseTool`）：

- 允许：Read / Grep / Glob（无限制）
- 允许：只读 Bash（`ls`，`find`，`grep`，`cat`，`stat`，`wc`，`head`，`tail` 等）
- 允许：Edit / Write **仅限 memory 目录内**
- 拒绝：Write-capable Bash（如 `>` 重定向、`|` 管道写文件）、MCP 工具、Agent 工具

PS：这个约束与 extractMemories 完全一致，通过 `createAutoMemCanUseTool()` 共享实现

17.5 DreamTask UI 集成

Auto-Dream 的 forked agent 运行在后台，对用户不可见——DreamTask 将其"可见化"到 footer pill 和 Shift+Down 浮动面板。

```
// DreamTaskState 结构
{
  type: 'dream',
  phase: 'starting' | 'updating',  // 收到 Edit/Write 后从 starting 切 updating
  sessionsReviewing: number,        // 要 review 的会话数
  filesTouched: string[],           // 观察到的文件路径（来自 tool_use，不完整）
  turns: DreamTurn[],               // 助手文本响应，工具调用折叠为计数
  abortController?: AbortController,
  priorMtime: number,              // 用于 kill 时回滚锁
}

// kill() 行为:
// 1. abort abortController（中止 forked agent）
// 2. status → killed
// 3. 回滚锁 mtime（rollbackConsolidationLock）
```

PS：filesTouched 是不完整的——只捕获 Edit/Write tool_use，不包含 bash中介的写入，所以它表示"至少改了这些"，而非"只改了这些"

17.6 GrowthBook 配置与开关

```
// autoDream.ts — getConfig() 读取 GrowthBook
' tengu_onyx_plover': {
  minHours: 24,       // 默认 24 小时
  minSessions: 5,     // 默认 5 个会话
}

// config.ts — isAutoDreamEnabled() 优先级
1. settings.json 的 autoDreamEnabled（用户显式设置）
2. GrowthBook 'tengu_onyx_plover'.enabled（实验组）
3. 返回 false
```

17.7 与 /dream 技能的关系

`/dream` 技能（`skills/bundled/dream.ts`）是手动触发的梦境命令，位于 `KAIROS` 或 `KAIROS_DREAM` feature flag 后：

- 手动触发时调用 `recordConsolidation()` 乐观地更新时间戳
- 同样使用相同的 consolidation prompt
- 同样是 forked agent，但走 /dream 技能路径，不经过 autoDream 的三门检查

---

18. ExtractMemories 系统（会话末记忆提取）

18.1 系统定位

ExtractMemories 是每轮对话结束时（`stopHooks`）触发的轻量记忆提取，通过 **forked agent** 运行。它不是主 agent 自己写记忆的替代品，而是补充机制——在主 agent 没有主动写记忆时，自动挖掘可提取的信息。

18.2 Forked Agent 模式

提取使用 `runForkedAgent()`，这是一个主对话的完美分支（perfect fork）：

- 相同的 system prompt（包含完整记忆保存指令）
- 共享父进程的 prompt cache
- 独立运行，不写回主 transcript（`skipTranscript: true`）
- 最多 5 个 turn（`maxTurns: 5`）防止验证 rabbit-hole

互斥机制（hasMemoryWritesSince）：
若主 agent 在本轮对话中已经写过记忆文件（通过检测 tool_use 是否包含 Edit/Write 到 autoMemPath），则跳过提取，光标直接前进。

```
// 互斥：主 agent 已写过 → 跳过提取
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  lastMemoryMessageUuid = lastMessage.uuid  // 光标推进，不处理
  return
}
```

18.3 闭包状态与 Trailing Run

提取模块通过 `initExtractMemories()` 创建闭包，捕获以下状态：

```
let lastMemoryMessageUuid        // 光标：已处理消息的最后一个 UUID
let inProgress = false           // 当前是否有提取正在运行
let turnsSinceLastExtraction = 0 // 节流计数器
let pendingContext               // 运行期间积压的上下文（用于 trailing run）
let inFlightExtractions = Set    // 所有未完成的提取 Promise
```

Trailing Run 机制：

- 若提取运行期间主循环又触发了一次提取 → 积压到 `pendingContext`
- 当前提取完成后，检查 `pendingContext`，执行 trailing run（跳过节流）
- 保证新消息不会漏掉

PS：

- "闭包状态"指的是：Agent 结束主对话后，提取记忆的后台任务**仍然引用着对话结束时的状态**（比如会话上下

文、pending 的操作等）。这些状态本应随会话关闭而清理，但因为提取任务还在跑，所以被"闭包"住了
- 本质就是：**后台任务持有着已结束会话的上下文引用，防止它被垃圾回收**

18.4 提交流水线

```
executeExtractMemories() 触发
  → 检查 GrowthBook gate（tengu_passport_quail）
  → 检查 isAutoMemoryEnabled()
  → 检查 inProgress（决定是否 stash trailing）
  → 调用 scanMemoryFiles() 获取 manifest（提前注入，避免 agent 白花一个 turn ls）
  → 构建提取 prompt（含 newMessageCount + existingMemories）
  → runForkedAgent({
      canUseTool: createAutoMemCanUseTool(memoryDir),
      maxTurns: 5,
      skipTranscript: true,
    })
  → 提取 writtenPaths
  → appendSystemMessage({ "Saved N memories" }) 到主对话
  → 刷新光标 lastMemoryMessageUuid

drainPendingExtraction() — 优雅关闭时调用
  → 等待所有 inFlightExtractions 完成（最多 60s）
  → 防止 shutdown 时杀掉正在运行的 forked agent
```

18.5 提取 Prompt 模板

两种 prompt 变体，由 `tengu_moth_copse` GrowthBook flag 控制是否跳过索引写入：

Auto-Only 变体（`buildExtractAutoOnlyPrompt`）：

- 只提取**个人私有记忆**，存到本地目录
- 告知主 agent：若用户要求记住某事，立即保存
- 给出四种类型（user/feedback/project/reference）定义
- 给出 "What NOT to save" 列表
- 两步保存：①写 topic 文件 ②更新 MEMORY.md (索引)

Combined 变体（`buildExtractCombinedPrompt`，team才有的功能）：

- 在 Auto-Only 基础上增加了**团队记忆**
- 额外包含每种类型的 `<scope>` 标签（私有 vs 团队）
- 额外要求：不要在团队记忆中保存敏感数据
- 两步保存：对私有和团队各自目录分别进行

---

19. DreamTask（梦境任务 UI 集成）

19.1 设计背景

Auto-Dream 的 forked agent 默认对用户不可见。DreamTask 通过任务注册框架（`registerTask` / `updateTaskState`）将其接入 UI，使后台进程可以在 footer pill 和 Shift+Down 对话框中展示。

19.2 任务状态结构

```
type DreamTaskState = {
  type: 'dream'            // 任务类型标识
  status: 'running' | 'completed' | 'failed' | 'killed'
  phase: 'starting' | 'updating'
  sessionsReviewing: number
  filesTouched: string[]  // 至少被修改过的文件路径（不完整）
  turns: DreamTurn[]      // 最近 30 个助手响应
  priorMtime: number      // 用于 kill 后回滚锁
}
```

19.3 消息监听（makeDreamProgressWatcher）

autoDream.ts 的 `makeDreamProgressWatcher` 监听 forked agent 的每个 assistant 消息：
每当 forked agent 发出一条 assistant 消息，立刻提取三样东西：

- 收集 text 块到 turn.text
- 统计 tool_use 块数量
- 从 Edit/Write tool_use 中提取 file_path，加入 filesTouched
  PS：因为 Auto Dream 是一个后台 fork 进程，用户看不到它的完整输出，但它会编辑文件，这里的操作是弄一份结构化的总结，告知用户"Auto Dream 做了什么"——实时收集进度，完成后汇总展示

19.4 杀任务（kill）

从 UI 终止 Dream 会：

1. `abortController.abort()` — 中止 API 调用
2. `rollbackConsolidationLock(priorMtime)` — 回滚锁 mtime，使下次可以重试

---

20. 每日日志模式（Assistant Mode / KAIROS）

20.1 背景

在 Assistant 模式下（perpetual session，区别于每次单独运行的 CLI 模式），**对话是持续不断的，不能依赖对话停顿点**。Claude Code 为此设计了**每日日志（Daily Log）** 模式。

20.2 与标准模式的区别

特性	标准模式（CLI）	每日日志模式（Assistant）
新记忆去向	直接写入 topic 文件	追加到 `logs/YYYY/MM/YYYY-MM-DD.md`
MEMORY.md 维护	每次写记忆后同步更新索引	暂时不维护（log 是 append-only）
索引整理	无	由 nightly /dream skill 定期将日志蒸馏为 topic 文件
触发条件	—	feature('KAIROS')

PS：

- 非 KAIROS 模式：用 Auto Dream（forked agent 后台跑记忆整合）
- KAIROS 模式：用 disk-skill dream，Auto Dream 被跳过

---

21. 启动链路与模块初始化

21.1 backgroundHousekeeping.ts

所有记忆模块在 `backgroundHousekeeping.ts` 中初始化（与 `initExtractMemories` 并行）：

```
export function startBackgroundHousekeeping(): void {
  // ① 始终初始化（无条件）
  void initMagicDocs()            // Magic Docs 服务
  void initSkillImprovement()     // 技能改进

  // ② 条件编译（EXTRACT_MEMORIES 未开启则 tree-shake 掉）
  if (feature('EXTRACT_MEMORIES')) {
    extractMemoriesModule!.initExtractMemories()
  }

  // ③ 始终初始化（在 extractMemories 之后）
  initAutoDream()

  // ④ 插件自动更新
  void autoUpdateMarketplacesAndPluginsInBackground()

  // ⑤ Deep link 注册（仅交互模式 + LODESTONE）
  if (feature('LODESTONE') && getIsInteractive()) {
    void registerProtocolModule!.ensureDeepLinkProtocolRegistered()
  }

  // ⑥ 延迟慢操作（10 分钟后执行）
  setTimeout(runVerySlowOps, 10min).unref()

  // ⑦ ant 用户：每 24 小时周期性清理
  if (process.env.USER_TYPE === 'ant') {
    setInterval(cleanupNpmCache + cleanupOldVersions, 24h).unref()
  }
}
```

backgroundHousekeeping.ts 是启动时并行拉起的**后台任务管家**：初始化记忆/文档/Dream 服务、调度清理任务，且全程不阻塞主流程启动

PS：

- 延迟慢操作（runVerySlowOps）：启动阶段用户如果已经在操作，不应该被清理任务抢占资源。等用户空闲了再跑
- .unref() — 不阻塞进程退出：这些定时器不计入 Node.js 的"活跃引用"。进程退出时不会被这些定时器卡住

21.2 stopHooks.ts 触发链路

```
// src/query/stopHooks.ts
// 每轮对话结束时（模型输出 final response，无 tool call）触发：
void executeExtractMemories(stopHookContext, appendSystemMessage)
void executeAutoDream(stopHookContext, appendSystemMessage)
```

两个系统均 fire-and-forget（不 await 等待完成），所以它们是并行非阻塞的。

21.3 关闭时 drain

```
// print.ts — 响应刷出后，gracefulShutdownSync 之前
await drainPendingExtraction(60_000)  // 最多等 60s
```

附录 A：Zod Schema 定义

A.1 Schema 定义

```
// teamMemorySync/types.ts
const TeamMemoryContentSchema = z.object({
  entries: z.record(z.string(), z.string()),           // key -> content
  entryChecksums: z.record(z.string(), z.string()).optional(),
})

const TeamMemoryDataSchema = z.object({
  organizationId: z.string(),
  repo: z.string(),
  version: z.number(),
  lastModified: z.string(),
  checksum: z.string(),
  content: TeamMemoryContentSchema(),
})

// agentMemorySnapshot.ts
const snapshotMetaSchema = z.object({
  updatedAt: z.string().min(1),
})

const syncedMetaSchema = z.object({
  syncedFrom: z.string().min(1),
})
```

附录 B：全局状态一览

B.1 各模块全局状态

会话记忆模块维护以下全局状态（`sessionMemoryUtils.ts`）：

```
let lastSummarizedMessageId: string | undefined = undefined
let extractionStartedAt: number | undefined = undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false
let sessionMemoryConfig: SessionMemoryConfig = { ...DEFAULT_SESSION_MEMORY_CONFIG }
```

Watch 模块维护以下全局状态（`watcher.ts`）：

```
let watcher: FSWatcher | null = null
let debounceTimer: ReturnType<typeof setTimeout> | null = null
let pushInProgress = false
let hasPendingChanges = false
let currentPushPromise: Promise<void> | null = null
```

```
let watcherStarted = false
let pushSuppressedReason: string | null = null
let syncState: SyncState | null = null
```

ExtractMemories 模块维护以下全局状态（`extractMemories.ts`，闭包作用域）：

```
let lastMemoryMessageUuid: string | undefined        // 光标：已处理的最后消息 UUID
let inProgress = false                               // 当前是否有提取在运行
let turnsSinceLastExtraction = 0                    // 节流计数器
let pendingContext: { context, appendSystemMessage } | undefined  // trailing run 积压上下文
let inFlightExtractions = new Set<Promise<void>>()  // 所有未完成的提取 Promise
let hasLoggedGateFailure = false                    // 是否已记录门禁用日志
```

AutoDream 模块维护以下全局状态（`autoDream.ts`，闭包作用域）：

```
let runner: ((context, appendSystemMessage?) => Promise<void>) | null = null
let lastSessionScanAt = 0  // 上次扫描时间（用于 SESSION_SCAN_INTERVAL_MS 节流）
```